

RNN (Recurrent Neural Network)

- ↳ type of network specially designed for sequential data
- It remembers previous info using memory (hidden state)
- send info from previous steps → next step so that they can learn patterns over time
- has feedback connection output from one step is reused in next step

current output = Function (current input + previous memory)

works

- Take current input (x_t)
- uses previous memory (h_{t-1})
- produces
 - new memory (h_t)
 - output (y_t)

memory is hidden state

$$h_t = \tanh \left(w_h \overset{\text{previous memory}}{h_{t-1}} + w_x \underset{\text{current input}}{x_t} \right)$$

$\downarrow$ current memory $\downarrow$ current input

Types

- Simple RNN
- LSTM
- GRU (Gated Recurrent Unit)
 - faster than LSTM, less complex, similar performance sometime

Problem

vanishing gradient $\rightarrow$ old info disappears

Exploding gradient $\rightarrow$ values become too large

Types

one $\rightarrow$ one

Basic Classification

one $\rightarrow$ many

Image $\rightarrow$ caption

many $\rightarrow$ one

Sentence $\rightarrow$ sentiment

many $\rightarrow$ many

Language Translation

Application

Language translation

chatbots

Speech Recognition

Text Generation

Sentiment Analysis

Hidden state $\rightarrow$ memory of ANN

store info of previous

inputs, timesteps

context of sequence

dependence

Current input x_i

previous hidden state h_{t-1}

Temporal dependency when current data

$\downarrow$
Time

depends on past data (time order matter)

$\rightarrow$ present value depend on past values

Ex

Weather depends on ~~past~~ Yesterday

Temp, humidity history, wind patterns

On model like RNN, LSTM, GRU, Transformer

Types

- Short term → Recent past matters more
 Ex → Last word in sentence
- Long term → Old past also matter
 Ex → Story context in paragraph

- Better sequence prediction, time series forecasting, NLP understanding
- Long short term memory.

LSTM Special type of RNN designed to remember info for a long time

- ⇒ solves problems of normal RNN forgetting old information
- ⇒ Works well with long sequences.
- ⇒ Resolves problem of vanishing gradient
- ⇒ With the help of memory cell and gates

Gates

1. Forget Gate → Decides what to remove from
 $b_t = \text{sigmoid}(w_f(h_{t-1}, x_t))$ memory Ex Remove irrelevant word

2. Input Gate → Decides what new info to store
 $i_t = \text{sigmoid}(w_i(h_{t-1}, x_t))$ Ex: → Store important context

3. Output Gate → Decides what to send as output
 $o_t = \text{sigmoid}(w_o(h_{t-1}, x_t))$ Ex Final prediction info.

Working

(cell state)

$$C_t = b_t C_{t-1} + i_t \tilde{C}_t$$

1. checks what to forget
2. Add new important info
3. Update memory
4. generate output

LSTM = RNN + Smart memory (control Gates).

next word prediction Predicting the most likely next word using previous words

⇒ Used in chat bots, Google search suggestion, AI text generation

1. Input words - reads previous words
2. Convert to Numbers - Words $\rightarrow$ Vectors
3. Context understanding
 RNN/LSTM $\rightarrow$ uses memory
 Transformer $\rightarrow$ uses attention
4. probability output

Encoder Decoder - Neural Network

architecture used to convert one sequence into another

Input sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ output sequence

Ex language translation

input	output
How are you	Tum Kaise ho

understand
input

Generate output

Encoder $\rightarrow$ understand English sentences

Decoder $\rightarrow$ Generates Hindi sentence

works

Step 1 encoder

- Reads input sequence word by word
 - Convert into context vector
- understanding sentence meaning

Step 2 - Context Vector

- stores compressed meaning of whole sentence
 - Passed to decoder
- ⇒ summary of input

Step 3 - decoder

- Take context vector
 - Generate output sequence step by step
- ⇒ forming translated sentence

Ex machine Translation

Text summarization

chatbots

Speech recognition

- RNN / LSTM Encoder - Decoder
 - older but important
- transformer Encoder - Decoder
 - modern and powerful (used in GPT, BERT, LLMs)

(*) single context vector → Info loss for long sentences
 solution → Attention mechanism

Context Vector = compressed summary of the entire input sequence

- Produced by Encoder
- Used by decoder to generate output

word

word 1 → update hidden state
 word 2 → —
 word 3 → —

Final hidden state → context vector

Ex
input

'I am learning AI'
 Encoder → creates → store context vector meaning

Decoder → uses it
 generates translation

memory at each step

If hidden state = memory

Then final memory = Context vector

Context vector = h_{final} Final summary vector /
memory of input
sequenceAI writing human like text
automatically2) NLG → Natural Language Generation↳ process of generating human like
text using AI

Data/Idea → AI model → human like structure

EX Chatbots

User: "What's weather today?"

Bot: "Today temp is 25°C and sunny."

works

- Content Planning → Decide what to say
- Sentence planning → Decide how to say
- Text Generation → Generate final sentence

models used

- RNN / LSTM → older text generation models
- Transformer → used in ChatGPT, Translation models

Application Chatbots

Voice Assistants

Auto Email Writing

Story Generation

Natural Language understanding (NLU)

↳ process where AI understands human language meaning
Human speaks/writes → AI understands → Take action

Ex Chatbot

User: Book ticket for tomorrow

NLU understands

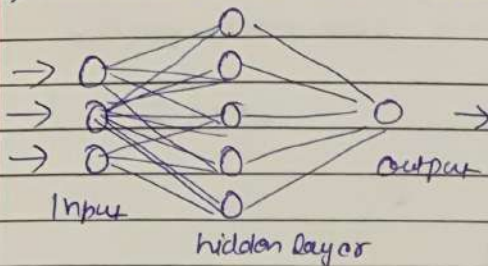
- Intent → Book ticket
- Entity → Date = Tomorrow

Work

1. Text processing → split sentence into words
2. Syntax understanding → Grammar + sentence structure
3. Semantic understanding → Actual meaning extraction
4. Intent + Entity Detection →
What user wants + important doc

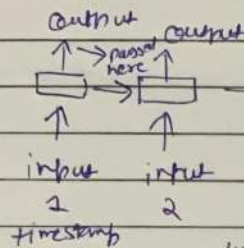
Feed forward $\rightarrow$ They cannot handle sequential data.

Neural networks are sequential



Recurrent Neural Networks

RNNs are recurrent
repeats again & again
over and over



$\rightarrow$ Each time steps has
two inputs except the
first one

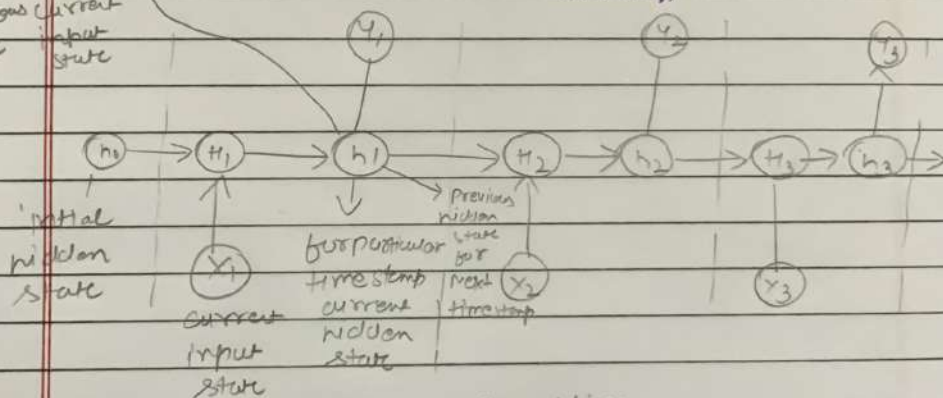
relative
timestamp

$$y_t = a [w_x x_t + w_y y_{t-1} + b]$$

w_x $\downarrow$ first timestamp
 w_y $\downarrow$ input from previous
timestamp

$h_t = f(h_{t-1}, x_t)$
 $\downarrow$
previous state
current input state

activation
function



first timestamp
second timestamp

job output

$$y_t = w_y h_t$$

$\downarrow$
weight
at output
state

To find
current hidden
state:

$$h_t = \tanh (w_{hh} h_{t-1} + w_{hx} x_t)$$

$\downarrow$ weight at previous state
 $\downarrow$ weight at current state

Source